

UNIT 4

Memory Management in Operating System

- Memory management is a critical aspect of operating systems that ensures efficient use of the computer's memory resources.
- It controls how memory is allocated and deallocated to processes, which is key to both performance and stability.
- When a program executes, it needs to be loaded into the **main memory (RAM)**. Since many programs run together, the **Operating System (OS)** manages how memory is allocated, used, and freed — this is called **Memory Management**.
- Memory management is the process of **allocating and deallocating** memory spaces to processes so that:
 - Memory is used **efficiently**.
 - Processes do not **interfere** with each other.
 - **CPU utilization** is maximized.

Objectives of Memory Management

1. Efficient Utilization:

Make maximum use of available memory space.

2. Fair Allocation:

Ensure each process gets the memory it needs.

3. Protection:

Prevent one process from accessing another's memory.

4. Support Multiprogramming:

Allow multiple processes in memory at the same time.

5. Minimize Fragmentation:

Reduce wasted space inside or between memory blocks.

Static and Dynamic Loading

Loading a process into the main memory is done by a loader. There are two different types of loading:

- **Static Loading:**
 - Static Loading is basically loading the entire program into a fixed address.
 - It requires more memory space.
- **Dynamic Loading:**
 - Dynamic loading loads program routines into memory only when they are needed.
 - This saves memory by not loading unused routines. The routines remain on disk in relocatable (can be loaded at any memory location) format until called.
 - It allows better memory utilization, especially for large programs.

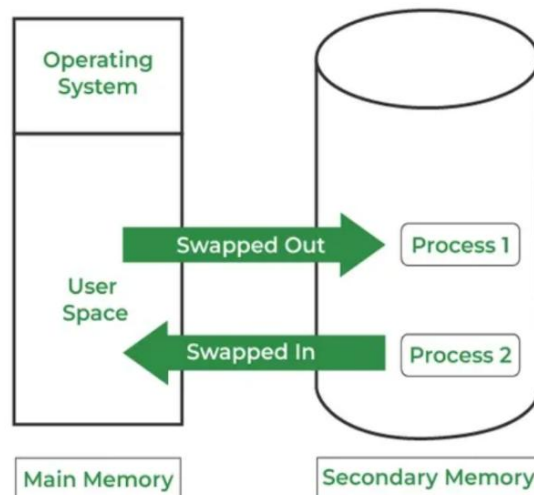
Static and Dynamic Linking

A **linker** combines object files into a single executable.

- **Static Linking:** All required modules are combined into one executable. No runtime dependency; some OS support only this method.
- **Dynamic Linking:** Uses a **stub** (small code) for library calls. At runtime, the stub checks if the routine is in memory; if not, it loads it.

Swapping

- **Swapping moves processes between main memory and secondary memory to manage limited memory space.**
- It allows multiple processes to run by temporarily swapping out lower priority processes for higher priority ones.
- The swapped-out process resumes once it's loaded back. Transfer time depends on the amount of data swapped.



Swapping

Binding of Instructions and Data to Memory

- Address binding refers to the process of associating program instructions and data with specific memory addresses.
- This can happen at **three different stages** of program execution:

1. Compile Time Binding

- If the memory location is known **a priori** (before execution), the **absolute code** can be generated.
- This means the compiler directly places physical addresses in the code.
- If the starting location of the program changes, the code must be **recompiled**.

Example:

A small embedded system where the program always loads at address 0x0000.

2. Load Time Binding

- Used when the memory location is **not known at compile time**.
- The compiler generates **relocatable code** instead of absolute code.
- The **loader** assigns the actual starting address when the program is loaded into memory.

Example:

When an OS loads multiple programs dynamically, each program is relocated to a suitable memory block.

3. Execution Time Binding

- Binding is delayed until **run time**.
- Required if a process can be **moved** during its execution (e.g., swapping between memory segments).
- Needs **hardware support** for address mapping — typically using **base and limit registers** or **MMU (Memory Management Unit)**.

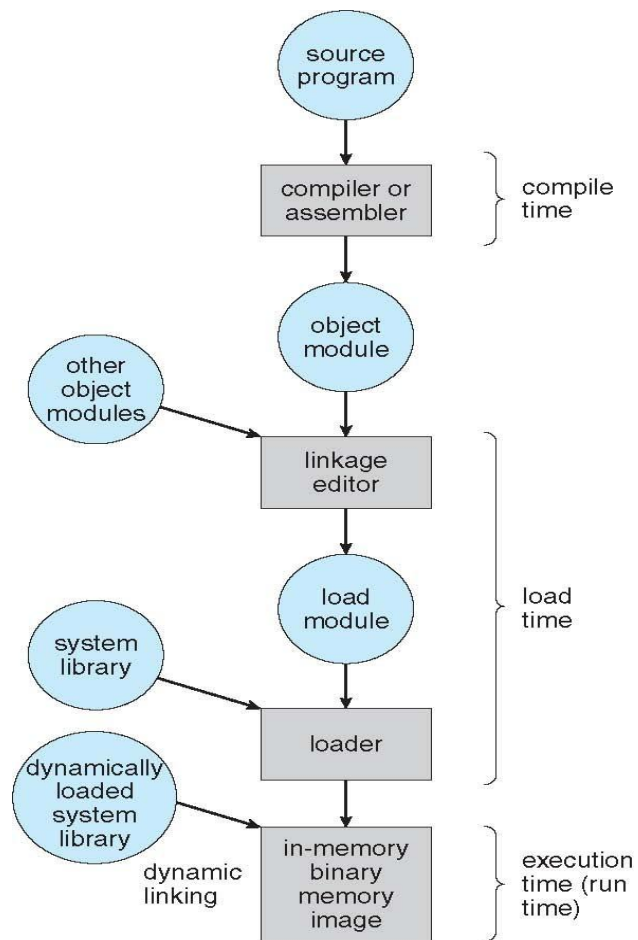
Example:

A process initially runs in one memory segment but is moved to another due to memory compaction — addresses are updated automatically by hardware.

Summary Table

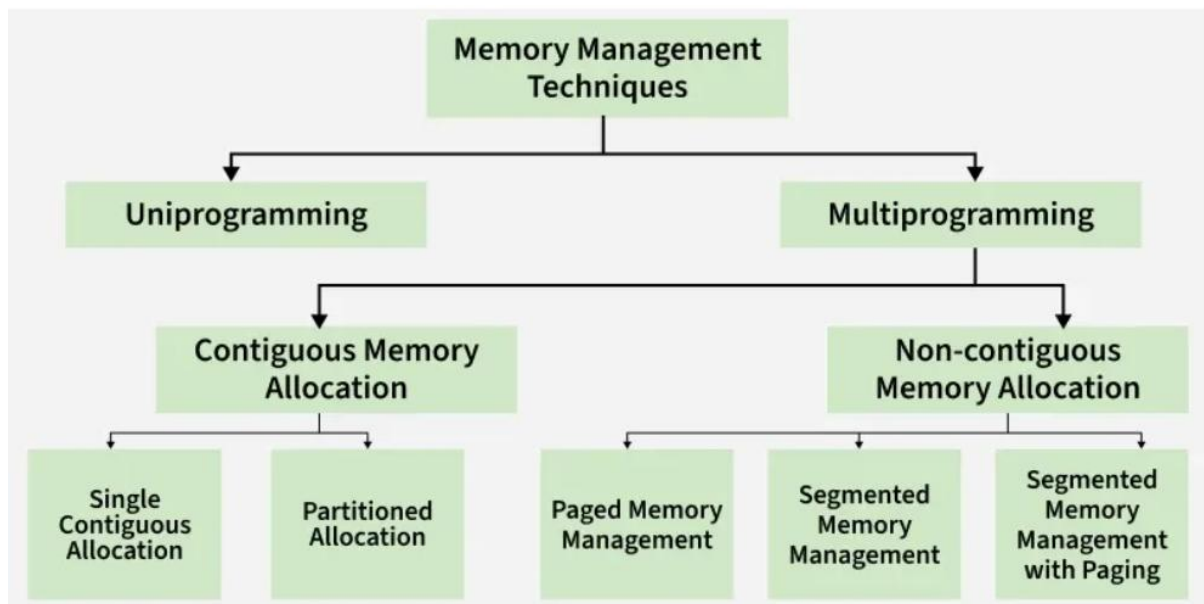
Binding Stage	When Performed	Type of Code	Flexibility	Example
Compile Time	Before execution	Absolute Code	Low	Embedded systems
Load Time	While loading	Relocatable Code	Moderate	Typical OS loading
Execution Time	During execution	Runtime Mapping	High	Virtual memory systems

- Execution-time binding enables **dynamic relocation**, allowing processes to move freely in memory while executing — this is essential for **modern multitasking operating systems**.



Memory Management Techniques

- Memory management techniques are **methods used by an operating system** to efficiently allocate, utilize, and manage memory resources for processes.
- Various techniques help the operating system manage memory effectively. They can be broadly categorized into:



Memory Management Techniques

Memory Management with Monoprogramming (Without Swapping)

- This is the simplest memory management approach the **memory is divided into two sections: One part of the operating system and the second part of the user program**
- The OS is loaded either at the bottom or the top of memory.
- Since interrupt vectors are usually placed in low memory, the OS is often loaded there.
- In a single-process system, sharing code or data is unnecessary.
- The OS is protected from user programs using a **fence register**.
- A **Fence Register** is a **hardware register** that stores a **memory address** which acts as a **boundary (fence)** between the **Operating System's memory area** and the **User Program's memory area**.
- **Example:**

Total Memory	512 MB
OS Reserved	100 MB
User Space	412 MB

- If a program of size 300 MB runs, 112 MB stays unused. If another process wants to run, it must wait till the first one finishes.
-
- **Limitation:**
No multiprogramming → CPU idle during I/O operations → Inefficient use.

Memory Management with Multiprogramming

1. Contiguous Memory Allocation
2. Non-Contiguous Memory Allocation

A. Contiguous Memory Allocation

- In this method, **each process occupies one continuous block** of memory space. Memory addresses used by a process are **consecutive**.
- Contiguous memory allocation is a memory management method where **each process is given a single, continuous block of memory**.
- This means all the data for a process is stored in adjacent memory locations.
- Thus, **whenever a process asks to access the main memory, we allocate a continuous segment from the empty region** to the process based on its size. In this technique, memory is allotted in a continuous way to the processes.

Contiguous Memory Management has two types:

- **Fixed Partitioning (Static)**
- **Variable Partitioning (Dynamic)**

a) Fixed Partitioning (Static Partitioning)

- Main memory is divided into **fixed-size** partitions.
- Each partition holds **exactly one process**.

Example:

Partition	Size (MB)	Process	Used	Wasted
P1	100	A (80)	80	20
P2	100	B (60)	60	40
P3	100	C (100)	100	0

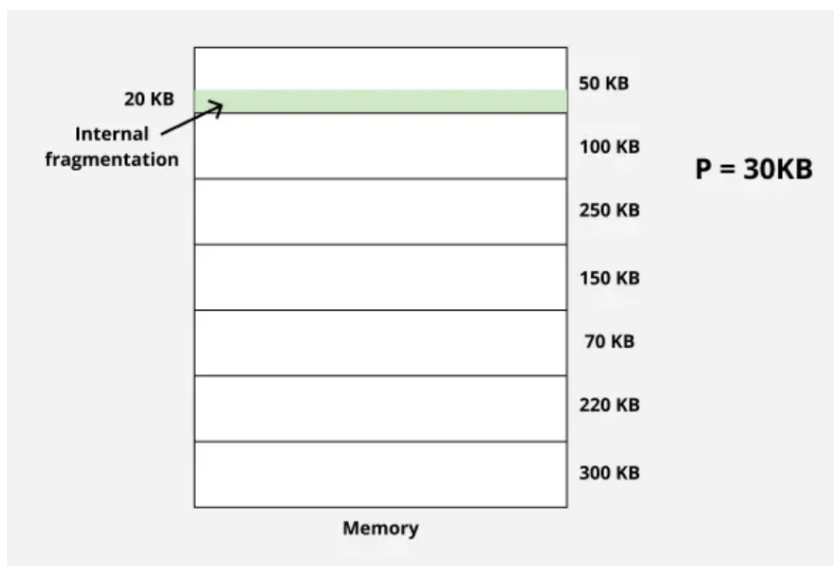
Internal Fragmentation = 20 + 40 = 60 MB

Advantages:

- Simple to implement.
- Easy to keep track of memory.

Disadvantages:

- **Internal Fragmentation:** Memory wasted within partitions.
- Number of processes limited by number of partitions.
- Poor flexibility.



b) Variable Partitioning (Dynamic Partitioning)

- Memory divided dynamically depending on process size.
- Each process gets exactly the amount of memory it needs.

Example:

Memory = 400 MB

Process A = 100 MB

Process B = 80 MB

Process C = 60 MB

Process D = 120 MB

Memory Layout:

| A(100) | B(80) | C(60) | D(120) | Free(40) |

After process B finishes, 80 MB hole remains → new process may or may not fit.

Problem:

When free memory is divided into small scattered parts → **External Fragmentation**

Solutions:

- **Compaction:** Move processes to merge free spaces into one large block.

Advantages:

- Better memory utilization.
- No internal fragmentation.

Disadvantages:

- External fragmentation possible.
- Compaction adds overhead.

Fragmentation(Very Important)

- When processes are loaded and removed from memory, **free memory space gets broken into small pieces**.
These small unusable memory spaces are called **fragments**, and the overall problem is known as **fragmentation**.
- It **reduces the efficiency of memory utilization**.

Types of Fragmentation

There are two main types:

1. **Internal Fragmentation**
2. **External Fragmentation**

1. Internal Fragmentation

- **Definition:**
Occurs when **fixed-sized memory blocks are allocated** to processes, but a process does **not use the entire block**.
- So, **unused memory inside an allocated partition** is wasted.

Example of Internal Fragmentation

Suppose memory is divided into **fixed partitions** of 200 KB each.

Process	Memory Required	Partition Allocated	Wasted Space
P1	180 KB	200 KB	20 KB
P2	90 KB	200 KB	110 KB
P3	200 KB	200 KB	0 KB

Total Internal Fragmentation = 20 + 110 + 0 = 130 KB

Formula for Internal Fragmentation:

Internal Fragmentation = Allocated Memory - Memory Requested

2. External Fragmentation

- Occurs when **free memory is available**, but **not in contiguous (continuous) blocks**.
- So, **a process cannot be allocated memory** even though total free space is enough.

Example of External Fragmentation

Suppose total memory = **500 KB**, divided as follows:

Block	Size	Status
1	100 KB	Free
2	200 KB	Allocated
3	50 KB	Free
4	100 KB	Allocated
5	50 KB	Free

Now, a new **process P4 (requires 150 KB)** comes in.

Total free memory = 100 + 50 + 50 = 200 KB (enough total space)

But no **single contiguous block of 150 KB** is available.

Hence, **P4 cannot be allocated**. This is **external fragmentation**.

Visualization

|100 Free|200 Used|50 Free|100 Used|50 Free|

Even though free = 200 KB, we can't fit 150 KB together, it's **scattered**.

Hence, memory **cannot be allocated** — external fragmentation occurs.

Difference Between Internal and External Fragmentation (Important)

Basis	Internal Fragmentation	External Fragmentation
Cause	Fixed partitioning	Variable partitioning
Memory wasted	Inside allocated partitions	Between allocated partitions
Type of free space	Inside allocated block	Between blocks
Can be removed by	Using variable partitioning	Using compaction or paging
Example	200 KB partition, process uses 180 KB	Free blocks scattered (100 KB + 50 KB + 50 KB)

How to Reduce Fragmentation

Technique	Description
Compaction	Rearranges memory contents to combine free spaces into one large block (used for external fragmentation).
Paging	Divides memory into fixed-size pages; eliminates external fragmentation.
Segmentation	Logical division of memory; can combine with paging to minimize fragmentation.

Compaction Example

Before compaction: |P1|Free(50)|P2|Free(30)|P3|Free(70)|

After compaction: |P1|P2|P3|Free(150)|

Now, the free space is **contiguous** — external fragmentation is removed.

Memory Allocation Strategies [Very Important]

These strategies determine **which free block (hole)** in memory should be assigned to an incoming process.

- Let's assume memory has the following **free holes** (in KB):

Hole 1 → 100 KB

Hole 2 → 500 KB

Hole 3 → 200 KB

Hole 4 → 300 KB

And a new **process** requires **212 KB**.

(i) First Fit

- Allocate the **first hole** that is **large enough** for the process.
- Search starts from the **beginning** of memory and stops at the first suitable space.

Example:

- Holes checked: **100**, 500 → allocate in Hole 2.
- Remaining free space in Hole 2 = $500 - 212 = 288$ KB.

Advantages:

- Simple and fast (less searching).
- Efficient for average workload.

Disadvantages:

- May cause small free holes near the beginning → **external fragmentation**.

(ii) Best Fit

- Allocate **the smallest hole** that is **large enough** to fit the process.
- Searches the **entire list** of holes before deciding.

Example:

- Holes large enough: 500, 300 → choose **300 KB (Hole 4)** since it's smallest that fits.
- Remaining free space = 88 KB.

Advantages:

- Minimizes wasted space **locally**.

Disadvantages:

- Creates **many small unusable holes** (external fragmentation).
- Slower (must search all holes).

(iii) Worst Fit

- Allocate the **largest available hole**.
- The idea is to **leave a large leftover hole**, which might fit future processes.

Example:

- Largest hole = 500 KB (Hole 2).
- Remaining free = $500 - 212 = 288$ KB.

Advantages:

- Fewer small holes (less fragmentation initially).

Disadvantages:

- Often wastes large spaces inefficiently.
- Poor overall memory utilization.

(iv) Next Fit

- Similar to **First Fit**, but instead of starting from the beginning every time, the **search continues from the point where the last allocation was made**.

Example:

- If last allocation was at Hole 2, start searching from Hole 3 next time.

Advantages:

- Avoids repeatedly filling lower memory.
- Distributes allocation more evenly.

Disadvantages:

- Still causes fragmentation.

Example

Free Memory (KB): 100, 500, 200, 300

Process Size: 212 KB

Method	Chosen Hole	Leftover (KB)
First Fit	500	288
Best Fit	300	88
Worst Fit	500	288
Next Fit	Depends on the last allocation	Var

Belongs To	Type	Purpose
Memory Management Technique:	Contiguous Allocation	Allocate continuous memory
Subcategory:	Variable Partitioning	Dynamic partition creation
Allocation Strategies:	First Fit, Best Fit, Worst Fit, Next Fit	Decide where to place new process

FIXED PARTITIONING WITH EQUAL AND UNEQUAL-SIZED PARTITIONS

Example 1: Fixed Partitioning (Equal Size Partitions)

Suppose **Main Memory = 1200 KB**

It is divided into **4 equal partitions of 300 KB each**.

Partitions:

Partition 1 = 300 KB

Partition 2 = 300 KB

Partition 3 = 300 KB

Partition 4 = 300 KB

Processes:

P1 = 250 KB

P2 = 90 KB

P3 = 310 KB (too large for any partition)

P4 = 120 KB

Allocation:

- P1 → Partition 1 (50 KB wasted → internal fragmentation)
- P2 → Partition 2 (210 KB wasted)
- P3 → Cannot be allocated (needs 310 KB, bigger than any partition)
- P4 → Partition 3 (180 KB wasted)
- Partition 4 → Empty (300 KB free, but only one process left and it's too big)

Internal Fragmentation = 50 + 210 + 180 = 440 KB wasted

Example 2: Fixed Partitioning (Unequal Size Partitions)

Suppose **Main Memory = 1200 KB**

It is divided into **unequal partitions** (decided at system startup, fixed and static):

Partitions:

Partition 1 = 100 KB

Partition 2 = 300 KB

Partition 3 = 400 KB

Partition 4 = 400 KB

Processes:

P1 = 90 KB

P2 = 250 KB

P3 = 380 KB

P4 = 420 KB (too large for any partition)

Allocation:

- P1 → Partition 1 (10 KB wasted)
- P2 → Partition 2 (50 KB wasted)
- P3 → Partition 3 (20 KB wasted)
- P4 → Cannot fit anywhere (needs 420 KB, but max partition = 400 KB)
- Partition 4 → Empty (400 KB free)

Internal Fragmentation = $10 + 50 + 20 = 80$ KB wasted

Example 3: Variable Size Partitioning (External Fragmentation)

- In **Variable Size Partitioning**, memory is divided **dynamically** as per the size of each incoming process.
- Each process occupies **exactly the space it needs**, no fixed partitions.
- This avoids **internal fragmentation** but can cause **external fragmentation** over time.

Given:

Let's assume **total main memory = 1000 KB**

Initially → all memory is free

[0 KB ----- 1000 KB]

(Entire 1000 KB free)

Now, processes arrive one by one:

Process	Size (KB)	Action
P1	200	Loaded
P2	300	Loaded
P3	100	Loaded
P4	200	Loaded

Step-by-Step Allocation

1. After loading P1, P2, P3, P4

Process	Start Address	End Address
P1	0	199
P2	200	499
P3	500	599
P4	600	799
Free space	800	999 (200 KB)

Memory Layout:

| P1(200) | P2(300) | P3(100) | P4(200) | Free(200) |

0 199 499 599 799 999

2. Now, P2 and P4 finish (terminated)

Memory after releasing P2 and P4:

Block	Size (KB)	Status
0–199	200	Occupied (P1)
200–499	300	Free
500–599	100	Occupied (P3)
600–799	200	Free
800–999	200	Free

Memory Layout:

| P1(200) | Free(300) | P3(100) | Free(200) | Free(200) |
0 199 499 599 799 999

3. New Process Arrives

New process **P5 = 350 KB**

Now, OS tries to find a contiguous free block of 350 KB.

- Free holes = 300, 200, 200
- None of these individual holes are large enough.
- **But total free memory = $300 + 200 + 200 = 700$ KB (enough in total!)**

However, **P5 cannot be loaded** because memory is **fragmented** into **non-contiguous** small blocks.

This is External Fragmentation

Even though **total free space = 700 KB**, it is divided into **three separate chunks (holes)**.

So, there is **enough total memory**,

but no single continuous block large enough to hold the process.

4. Solution – Compaction

To fix external fragmentation, the OS performs **Compaction**:

- It moves occupied blocks together.
- Merges all free spaces into **one large block**.

After compaction:

| P1(200) | P3(100) | P5(350) | Free(350) |

Now, **P5 (350 KB)** fits easily, and **the remaining free space = 350 KB** (continuous).

PRACTICE QUESTIONS

Question 1

Memory partitions (in KB): 100, 500, 200, 300, 600

Processes (in KB): 212, 417, 112, 426

Allocate memory using:

- a) First Fit
- b) Best Fit
- c) Worst Fit

- (i) Show step-by-step allocation and indicate which process remains unallocated (if any).
- (ii) Calculate the percentage of memory utilization.

Question 2

Memory blocks: 50 KB, 200 KB, 300 KB, 350 KB, 600 KB

Processes: 115 KB, 500 KB, 358 KB, 200 KB

Find allocation using:

- a) First Fit
- b) Next Fit

Assume pointer starts from the first block.

B. Non-Contiguous Memory Allocation

- Here, a process is **divided into parts** that can be placed **anywhere** in memory (not necessarily adjacent).
- The OS uses mapping structures to keep track of each part.

Techniques:

1. Paging
2. Segmentation
3. Paged Segmentation (Hybrid)

Logical and Physical Address Space

When a program runs, it needs to access memory (instructions, data, variables). However — programs don't directly deal with *real (physical)* memory addresses. Instead, they use *logical addresses* generated by the **CPU**.

So, there are **two types of address spaces** in memory management:

Type	Generated By	Visible To	Example
Logical Address	CPU	Program (user)	0x00A1
Physical Address	Memory Unit (RAM)	Hardware only	0x2FA1

Logical Address (Virtual Address):

- Address generated by the **CPU** during program execution.
- It is part of the **logical address space** — a set of all logical addresses a program can use.
- These are **used by programs** (not actual physical RAM locations).
- Mapped to physical addresses by the **Memory Management Unit (MMU)**.

Example:

Program refers to variable X at logical address **1000**.

Physical Address:

- Actual location in the **main memory (RAM)**.
- Generated after address translation by **MMU**.
- CPU never directly uses physical addresses — it uses logical ones that are converted.

Example:

Logical address **1000** → Physical address **5500**

Address Translation (Role of MMU)

The **Memory Management Unit (MMU)** converts logical addresses to physical addresses using a **base register** (also called a relocation register).

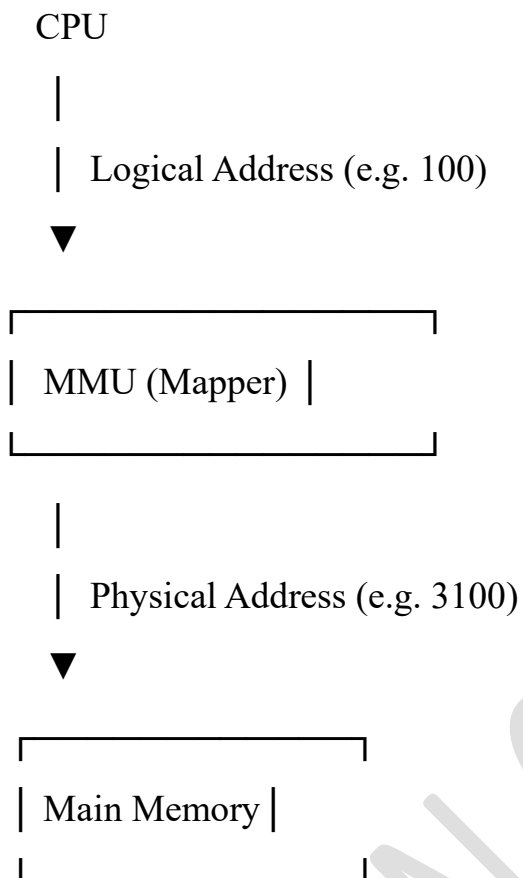
Formula:

Physical Address = Base Address + Logical Address

Example:

Base Register	Logical Address	Physical Address
3000	100	3100
3000	500	3500

Diagram



Example

Given:

Base (relocation) register = 1000

Logical addresses = 120, 350, 800

Find physical addresses:

Logical	Base	Physical
120	1000	1120
350	1000	1350
800	1000	1800

Key Differences

Feature	Logical Address	Physical Address
Generated by	CPU	Memory Unit (MMU)
Visible to	User / Program	OS only
Changes with relocation	Yes	No
Example	1200	4200
Used in	Virtual memory, paging	Actual data storage

Real-life Analogy

- **Logical address** = Apartment number on your mail
- **Physical address** = Actual street address where the apartment is located
The **mailman (MMU)** maps apartment numbers to physical locations.

Relocation Register

- A **Relocation Register** (also called a **Base Register**) is a special hardware register used in memory management to **map logical addresses to physical addresses** during program execution.
- It helps the operating system **relocate** (move) a program in memory **without changing its code**.

Need

- When a program is loaded into memory, it can be placed **anywhere** — not always starting from address 0. But the program's instructions and variables are written **as if it starts from 0** (logical address).
- The **Relocation Register** adds the **starting physical address** of that program to each logical address generated by the CPU.

This ensures:

- Program executes correctly, even if moved.
- No need to modify instruction addresses inside the program.

Working

Each process is given two registers:

1. **Relocation Register (Base Register)** → Holds the starting physical address of the process.
2. **Limit Register** → Holds the length (size) of the process, to prevent it from accessing outside memory.

Formula:

Physical Address = Relocation Register + Logical Address

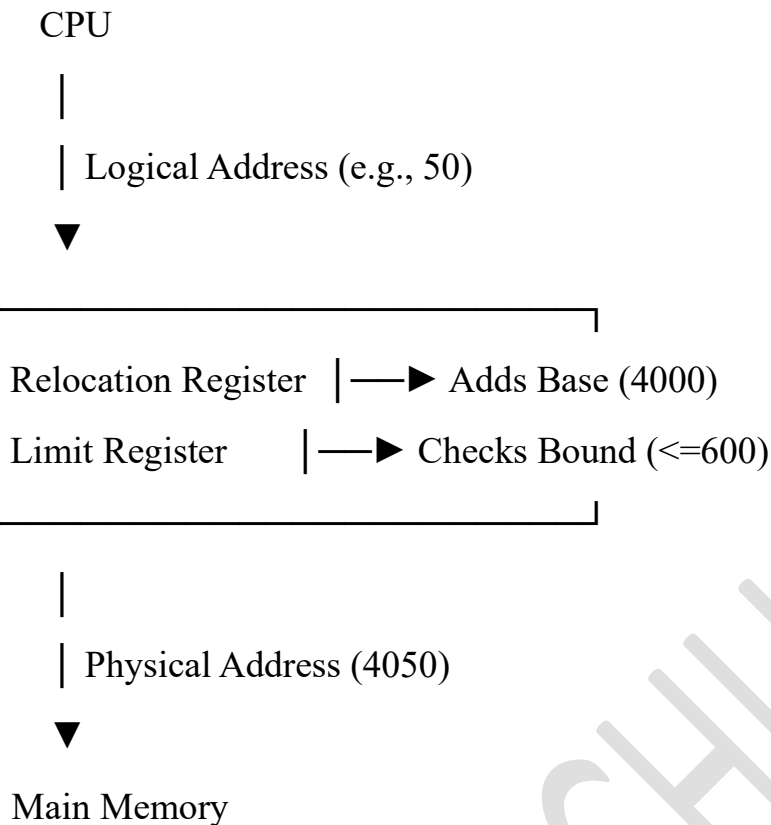
If Logical Address $\geq$ Limit Register → Trap (Memory Violation Error)

Example

Process	Relocation Register (Base)	Limit	Logical Address	Physical Address
P1	4000	600	50	4050
P2	10000	1200	300	10300

Explanation:

- For **P1**, if CPU generates logical address 50, actual (physical) address = $4000 + 50 = 4050$
- If logical address > 600 → memory access error (out of range)



Non Contiguous Memory Allocation

Non-contiguous allocation involves the use of pointers to link the non-contiguous memory blocks allocated to a process. These pointers are used to keep track of the memory blocks allocated to the process and to locate them during the execution of the process.

- **Paging:** It breaks processes into equal-sized pages and maps them to memory frames, mainly to eliminate external fragmentation.
- **Segmentation:** It divides processes into variable-sized logical segments (like code, data, stack), supporting protection, sharing, and logical organization.
- **Segmented Paging:** A hybrid technique that first divides processes into logical segments and then splits each segment into fixed-size pages, combining the benefits of segmentation and paging.

It has the advantage of reducing memory wastage but it increases the overheads due to address translation, which results in slow execution.

Note: Paging avoids external fragmentation but may suffer from internal fragmentation.

How Does Non-Contiguous Memory Allocation Work?

A process is divided into parts that can be stored in different free spaces of main memory (not necessarily consecutive).

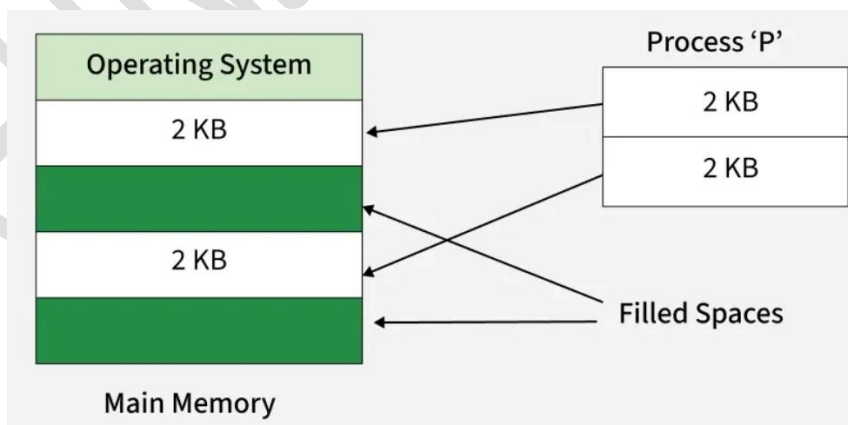
Example: If a process $P = 4KB$ and memory has two free slots of $2KB$ each, contiguous allocation fails (no spanning allowed). In non-contiguous allocation, P is split into two $2KB$ parts, each fitting into the free slots.

In contiguous allocation:

- Space in memory should be allocated to the whole process.
- If not, then that space remains unallocated.

In Non-Contiguous allocation:

- The process can be divided into different parts hence filling the space in the main memory.
- In this example, process P can be divided into two parts of equal size - $> 2KB$.
- Hence one part of process P can be allocated to the first $2KB$ space of main memory and the other part of the process can be allocated to the second $2KB$ space of main memory.



In what manner we divide a process to allocate them into main memory?

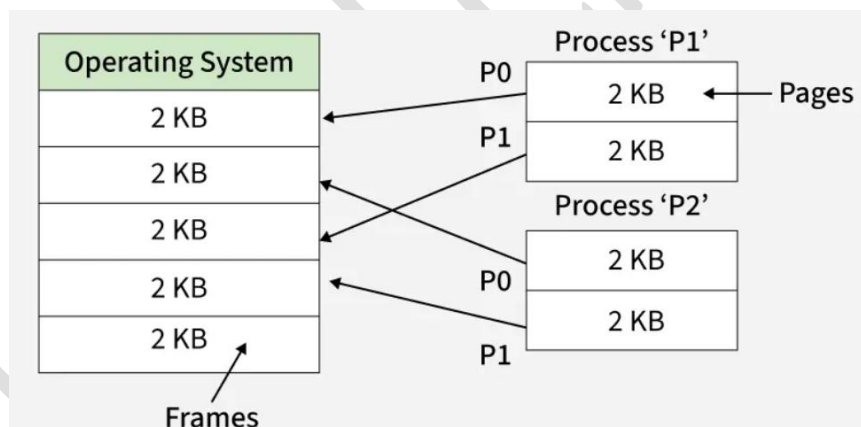
The process is divided after analyzing the number of empty spaces and their size in the main memory. But, it has some challenge:

Dividing processes dynamically based on changing free space is time-consuming as their sizes change every time due to the execution of already present processes in the main memory.

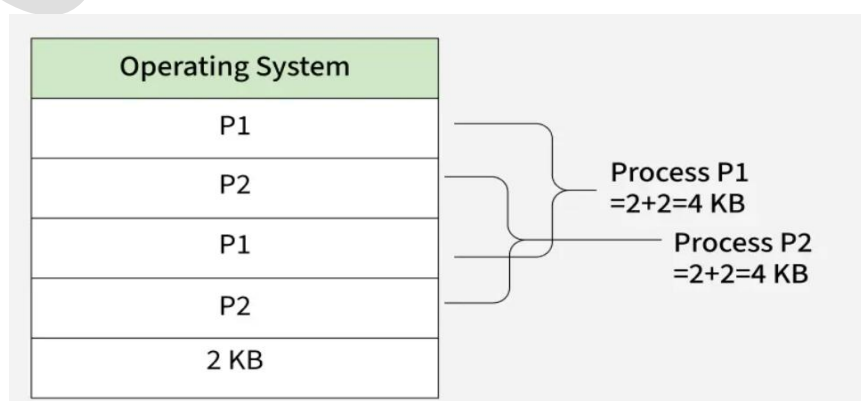
- In order to avoid this time-consuming process, we divide our process in secondary memory in advance before reaching the main memory for its execution.
- Every process is divided into various parts of equal size called Pages.
- We also divide our main memory into different parts of equal size called Frames. It is important to understand that:

Rule: $\text{pageSize} = \text{frameSize}$.

Example with Paging: If frame size = 2KB, processes P1 and P2 (2 pages each) can be stored alternately across free frames in memory.



Resolvent main memory, In main memory the sequence of storage of pages are:
First page of P1 -> First page of P2 -> Second page of P1 -> Second page of P2



Pros of Non-Contiguous Allocation

- **Reduced Internal Fragmentation** : Memory blocks are allocated as per process needs; unused space inside blocks is minimized.
- **Flexible Allocation** : Processes can be loaded wherever free memory is available, without requiring a large continuous block.
- **Better Memory Utilization** : Small gaps in memory can be efficiently used to fit parts of different processes.
- **Supports Dynamic Loading & Growth** : Processes that expand (like stack/heap) can take memory from scattered free blocks.
- **Multiprogramming Support** : More processes can reside in memory simultaneously, as allocation is more flexible.
- **Enables Virtual Memory** : Forms the basis of schemes like paging and segmentation, allowing processes to use more memory than physically available.

Cons of Non-Contiguous Allocation

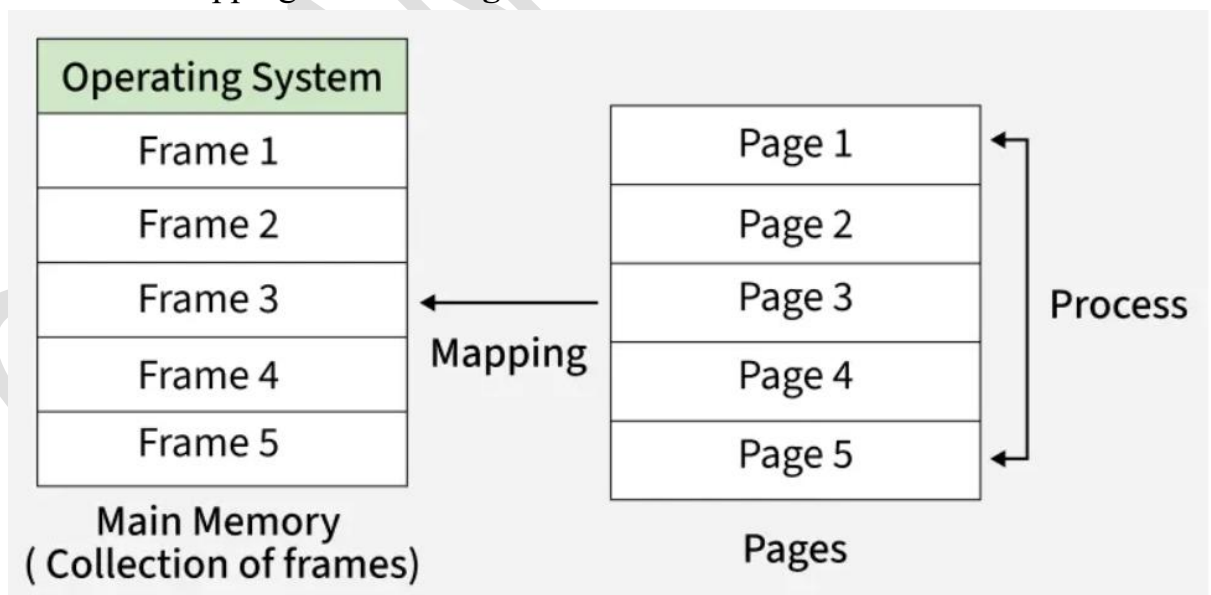
- **External Fragmentation** : Free memory may be broken into many small scattered blocks, making it hard to allocate large memory requests.
- **Overhead of Address Mapping** : Requires complex data structures (page tables, segment tables or linked lists) to keep track of allocated blocks.
- **Slower Access** : Address translation and pointer chasing increase access time compared to contiguous allocation.
- **Memory Management Overhead** : Additional hardware (MMU : Memory Management Unit) and OS routines are needed for mapping logical to physical addresses.
- **Complex Deallocation** : Releasing and merging free memory blocks is more complicated, increasing OS overhead.
- **Possibility of Page Faults (in Paging-based systems)** : Since data is scattered, a needed page may not be in memory, causing delays.

Paging in Operating System

- **Paging is the process of moving parts of a program, called pages, from secondary storage into the main memory (RAM).**
- The main idea behind paging is to break a program into smaller fixed-size blocks called pages.
- **Process does not have to be allocated in a contiguous memory space.**
- The whole process does not have to be in main memory; some pages can be present, and some pages can be loaded when needed. **It allows more processes and even processes larger than main memory to run.**
- **Memory allocation is simplified as memory is always allocated in fixed sized pages.**

Page Table

- **To keep track of where each page is stored in memory, the operating system uses a page table.**
- This table shows the connection between the logical page numbers and the physical page frames (actual locations in RAM).
- Maintains mapping between **Page Number** and **Frame Number**



Thus,

Paging divides both physical memory and process memory into **fixed-size blocks**:

- **Physical Memory → Frames**

- **Logical Memory (Process) → Pages**

Size of page = size of frame

When a process loads, its pages are placed into any available frames.

Example:

Page size = 1 KB

Process size = 4 KB → divided into 4 pages (P0, P1, P2, P3)

Physical memory has free frames F2, F5, F8, F9

→ Pages can go to any free frames

Page Table:

Page No.	Frame No.
0	2
1	5
2	8
3	9

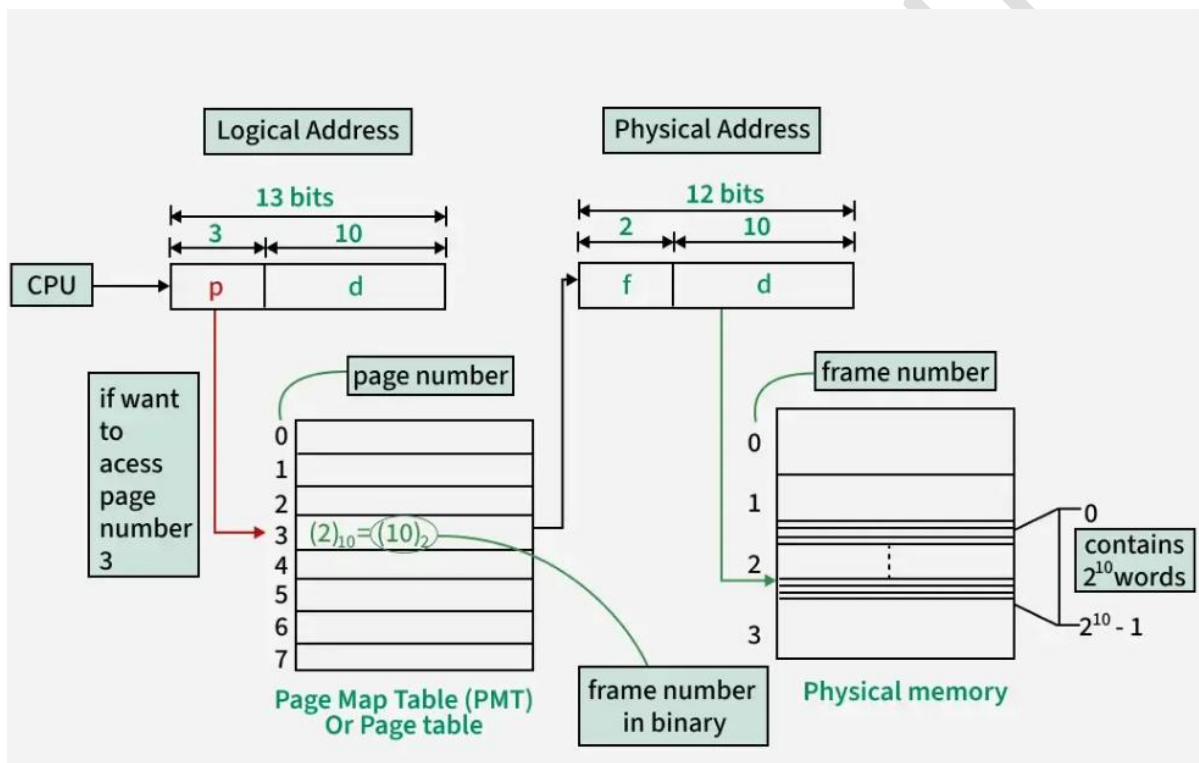
Important Features of Paging

- **Logical to physical address mapping:** Paging divides a process's logical address space into fixed-size pages. Each page maps to a frame in physical memory, enabling flexible memory management.
- **Fixed page and frame size:** Pages and frames have the same fixed size. This simplifies memory management and improves system performance.
- **Page table entries:** Each logical page is represented by a page table entry (PTE). A PTE stores the corresponding frame number and control bits.
- **Number of page table entries:** The page table has one entry per logical page. Thus, its size equals the number of pages in the process's address space.

- **Page table stored in main memory:** The page table is kept in main memory. This can add overhead when processes are swapped in or out.

Working of Paging

- When a process requests memory, the operating system allocates one or more page frames to the process and maps the process's logical pages to the physical page frames.
- When a program runs, its pages are loaded into any available frames in the physical memory.



Each program has a page table, which the operating system uses to keep track of where each page is stored in physical memory.

When a program accesses data, the system uses this table to convert the program's address into a physical memory address.

Steps Involved in Paging :

- **Step 1 Divide Memory :** Logical -> Pages, Physical -> Frames .
- **Step 2 Allocate Pages :** Load pages into available frames.
- **Step 3 Page Table :** Map logical pages to physical frames.
- **Step 4 Translate Address :** Convert logical to physical address.

- **Step 5 Handle Page Fault** : Load missing pages from disk.
- **Step 6 Run Program** : CPU uses page table during execution.

Example

- **Physical Address** = 12 bits, then Physical Address Space = 4 K words
- **Logical Address** = 13 bits, then Logical Address Space = 8 K words
- **Page size** = frame size = 1 K words (assumption)

Number of frames = Physical Address Space / Frame Size = $4K / 1K = 4 = 2^2$

Number of Pages = Logical Address Space / Page Size = $8K / 1K = 2^3$

The address generated by the CPU is divided into:

1. **Page number(p)**: Number of bits required to represent the pages in Logical Address Space or Page number
2. **Page offset(d)**: Number of bits required to represent a particular word in a page or page size of Logical Address Space or word number of a page or page **offset**.

A Physical Address is divided into two main parts:

1. **Frame Number(f)**: Number of bits required to represent the frame of Physical Address Space or Frame number frame
2. **Frame Offset(d)**: Number of bits required to represent a particular word in a frame or frame size of Physical Address Space or word number of a frame or frame offset.

Main memory access time = m,

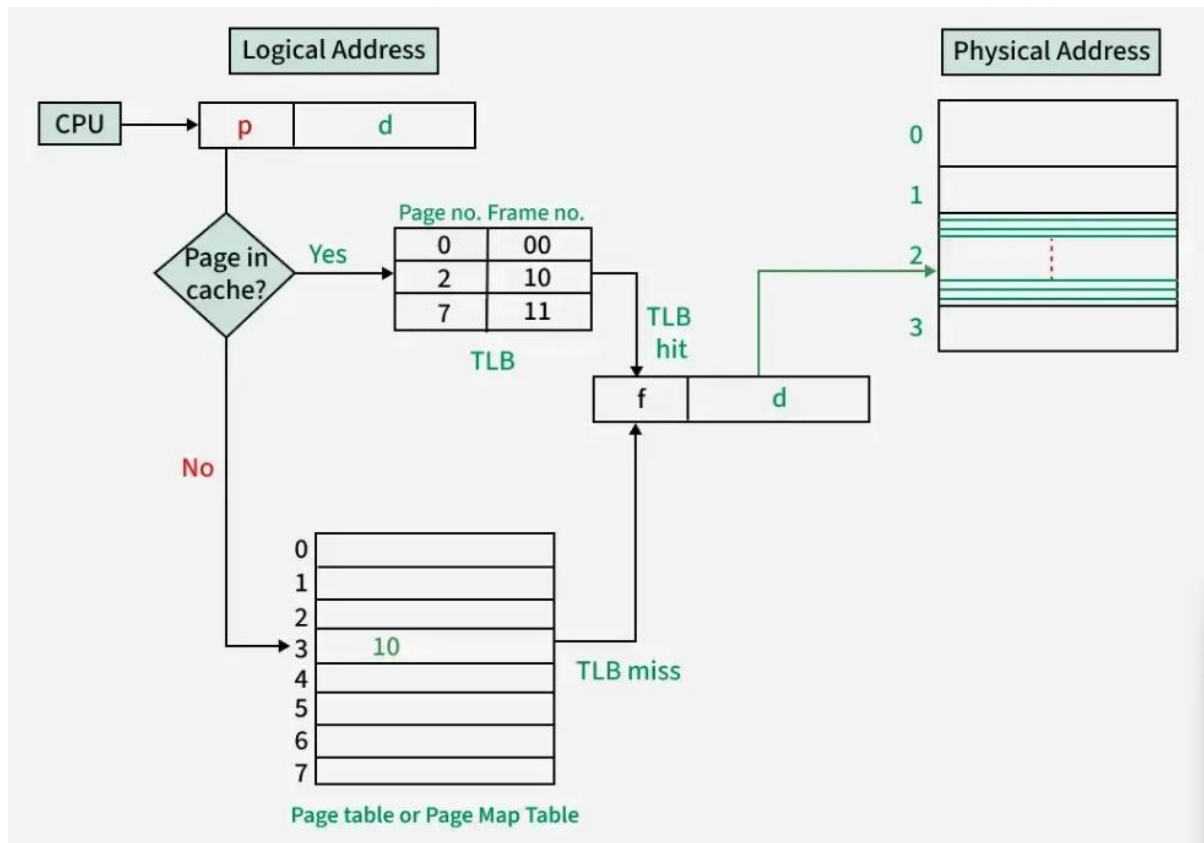
If page tables are kept in main memory, [Without TLB]

Effective access time = m(for page table) + m(for particular page in page table)

Therefore, EAT=2*m

Hardware Implementation of Paging

- The hardware implementation of the page table can be done by using dedicated registers. But the usage of the register for the page table is satisfactory only if the page table is small.
- If the page table contains a large number of entries then we can use TLB(translation Look-aside buffer), a special, small, fast look-up hardware cache.



What is TLB?

- **TLB (Translation Lookaside Buffer)** is a **special high-speed cache** used in the **Memory Management Unit (MMU)**.
- It stores **recent translations of virtual addresses to physical addresses**.
- It's like a **short-term memory** for the CPU to remember recent page table lookups.
- Each entry in TLB consists of two parts: a tag and a value.

- When this memory is used, then an item is compared with all tags simultaneously. If the item is found, then the corresponding value is returned.

Why is TLB Needed?

Paging requires **address translation**:

- CPU generates a **virtual address (VA)**.
- OS maintains a **page table** to map virtual pages to physical frames.
- To get the physical address, the system must **look up the page table**.

This lookup takes **extra memory access time**.
So, **each memory access might require 2 accesses**:

1. One for **page table lookup**.
2. One for **actual data**.

This slows down performance.

- **TLB reduces this time** by keeping recent page-table entries in fast memory (cache).

How TLB Works

1. CPU generates a **virtual address** → split into:
 - **Page number**
 - **Offset**
2. **MMU checks the TLB**:
 - If **page number is found (TLB Hit)** → get frame number directly from TLB.
 - If **page number not found (TLB Miss)** → lookup in **page table** (main memory).
3. Once found in page table → update TLB for faster future access.

Performance Impact

Let's calculate how TLB improves effective memory access time (EAT):

Formula for Effective Access Time (EAT)

$$EAT = (h) \times (t_{cache} + t_{memory}) + (1 - h) \times (t_{cache} + 2 \times t_{memory})$$

Where:

- h = TLB Hit Ratio (0 to 1)
- t_{cache} = Time to access TLB
- t_{memory} = Time to access main memory

Advantages of Paging

- **Eliminates External Fragmentation:** Paging divides memory into fixed-size blocks (pages and frames), so processes can be loaded wherever there is free space in memory. This prevents wasted space due to fragmentation.
- **Efficient Memory Utilization:** Since pages can be placed in non-contiguous memory locations, even small free spaces can be utilized, leading to better memory allocation.
- **Supports Virtual Memory:** Paging enables the implementation of virtual memory, allowing processes to use more memory than physically available by swapping pages between RAM and secondary storage.
- **Ease of Swapping:** Individual pages can be moved between physical memory and disk (swap space) without affecting the entire process, making swapping faster and more efficient.
- **Improved Security and Isolation:** Each process works within its own set of pages, preventing one process from accessing another's memory space.

Disadvantages of Paging

- **Internal Fragmentation:** If the size of a process is not a perfect multiple of the page size, the unused space in the last page results in internal fragmentation.
- **Increased Overhead:** Maintaining the Page Table requires additional memory and processing. For large processes, the page table can grow significantly, consuming valuable memory resources.
- **Page Table Lookup Time:** Accessing memory requires translating logical addresses to physical addresses using the page table. This step increases memory access time, although TLBs can help reduce the impact.
- **I/O Overhead During Page Faults:** When a required page is not in physical memory (page fault), it needs to be fetched from secondary storage, causing delays and increased I/O operations.
- **Complexity in Implementation:** Paging requires sophisticated hardware and software support, including the MMU and algorithms for page replacement, which add complexity to the system.

Numerical Questions on Paging, LAS & PAS

[Separate PDF]

Numerical Questions on Effective Memory Access Time

Example 1:

Given:

- Memory access time = **100 ns**
- TLB access time = **10 ns**
- TLB hit ratio = **90% (0.9)**

Find **EAT**.

When **TLB hit** →

= TLB access + memory access

= 10 + 100 = **110 ns**

When **TLB miss** →

= TLB access + (page table lookup + memory access)

= 10 + (100 + 100) = **210 ns**

Now,

$$EAT = (0.9)(110) + (0.1)(210)$$

$$EAT = 99 + 21 = 120 \text{ ns}$$

Effective Access Time = 120 ns

So, performance improves from 200 ns → 120 ns.

Example 2:

If hit ratio = **98%**, what is EAT?

$$EAT = (0.98)(110) + (0.02)(210)$$

$$EAT = 107.8 + 4.2 = 112 \text{ ns}$$

VERY IMPORTANT

Case	Steps Involved	Total Access Time
Without TLB	Page table lookup + Memory access	$2 \times 100 = 200 \text{ ns}$
With TLB (Hit)	TLB + Memory access	$10 + 100 = 110 \text{ ns}$
With TLB (Miss)	TLB + (Page table + Memory)	$10 + 200 = 210 \text{ ns}$

Example 3

Main memory access time = 80 ns.

There is no TLB used.

Each address translation needs one page table lookup and one memory access.

Find the **effective memory access time (EAT)**.

Solution:

$$EAT = \text{Page table Access} + \text{Memory Access}$$

$$EAT = 80 + 80 = 160 \text{ ns}$$

Example 4:

Given:

- Memory access time = 100 ns
- TLB access time = 10 ns
- TLB hit ratio = 95%

Find **effective memory access time**.

Solution:

$$EAT = h(10 + 100) + (1 - h)(10 + 200)$$

$$EAT = 0.95(110) + 0.05(210)$$

$$EAT = 104.5 + 10.5 = \boxed{115 \text{ ns}}$$

Example 5:

Given:

- Memory access time = 80 ns
- TLB access time = 5 ns
- Hit ratio = 98%

Find **EAT**.

Solution:

$$EAT = 0.98(5 + 80) + 0.02(5 + 160)$$

$$EAT = 0.98(85) + 0.02(165)$$

$$EAT = 83.3 + 3.3 = \boxed{86.6 \text{ ns}}$$

Case	What happens	Accesses	Total time
TLB Hit	TLB + Memory	2 accesses (fast)	$t_t + t_m$
TLB Miss	TLB + Page Table + Memory	3 accesses	$t_t + 2t_m$